P2956R2
Allow `std::simd` overloads for saturating operations

Published Proposal, 2026-03-30

This version:
http://wg21.link/P2956R2

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to add `std::simd` overloads for the saturating arithmetic operations introduced in [P0543R3].

Table of Contents

§ 1. Revision History

R1 => R2

- Updated to use new names for functions (e.g., `saturating_add` instead of `add_sat`).
- Updated to latest names from working draft (e.g., `basic_vec`)
- Fixed some rendering issues.

R0 => R1

- Added implementation experience.
- Improved wording.

§ 2. Motivation

The Working Draft of C++26 includes data parallel types. It mostly provides operators which work on or with `std::simd` types, but it also includes overloads of useful functions from other parts of C++ (e.g., sin, cos, abs). In [P0543R3] a proposal was made to provide saturating operation support for some basic arithmetic operations and casts. In particular, `saturating_add`, `saturating_sub`, `saturating_mul` and `saturating_div` are provided. These perform saturating arithmetic operations which are effectively performed in infinite precision, and will return the smallest or largest value when it is too large to be represented in that type. In addition, `saturating_cast` is also provided to convert to a new type, and to saturate to the range of that type if required.

These saturating functions should be provided in `std::simd` as element-wise operations.

> NOTE: Previous versions of this proposal used different names, which were updated as per NB comment FR-026-265.

§ 3. Implementation Experience

The most common types of saturating operations are addition, subtraction, and casting. All three of these functions have been implemented in Intel's reference implementation and used in our software products. Where hardware support is available for a data type these functions compile into native instructions (e.g., 16-bit integer saturations compile into `vpaddsw`, `vpsubsw`, and `vpmovsdw` respectively). For data types which have no saturating support in the hardware for those three functions (e.g., large integers) the compiler can generate efficient code to perform the operation (in the case of LLVM the `builtin_add_sat` function is used to hand this task to the compiler, rather than having the library itself generate the required code sequence). Examples of native versus non-native instruction sequences are given here:

Source	Output from clang 20

`// 16-bit saturating add`
`// native instruction`
`auto r16 = saturating_add(x16, y16);`
	`vpaddsw %zmm1, %zmm0, %zmm0`

`// 32-bit -> 16-bit saturating convert`
`// Native instruction`
`auto r16 = saturating_cast<int16_t>(x32);`
	`vpmovsdw %zmm0, %ymm0`

`// 32-bit saturating add`
`// Non-native (synthesised)`
`auto r16 = saturating_add(x32, y32);`
	`vpaddd  %zmm1, %zmm0, %zmm2`
	`vpcmpgtd  %zmm2, %zmm0, %k0`
	`vpmovd2m  %zmm1, %k1`
	`kxorw  %k0, %k1, %k1`
	`vpsrad  $31, %zmm2, %zmm0`

The other saturating operations haven't been implemented in the reference software as they are rarely needed. However, they can be trivially implemented in terms of the existing draft C++26 support for scalar saturating operations, or an optimized equivalent can be synthesized.

§ 4. Wording

§ 4.1. Modify [**version.syn**]

In [version.syn] bump the `__cpp_lib_simd` version.

§ 4.2. Modify [**simd.syn**]

In the header `<simd>` synopsis - [simd.syn] - add at the end after the "Complex Math" functions.

```
template<simd-floating-point V>
  rebind_t<complex<typename V::value_type>, V> polar(const V& x, const V& y = {

template<simd-complex V> constexpr V pow(const V& x, const V& y);

// [simd.saturating.math], saturating math functions
template<vec-simd-type V> constexpr V saturating_add(const V& x, const V& y) no
template<vec-simd-type V> constexpr V saturating_sub(const V& x, const V& y) no
template<vec-simd-type V> constexpr V saturating_mul(const V& x, const V& y) no
template<vec-simd-type V> constexpr V saturating_div(const V& x, const V& y) no
template<class U, vec-simd-type V>
  constexpr rebind_t<U, V> saturating_cast(const V& v) noexcept;
```

Add the following to the end of the using declarations:

```
// See [simd.complex.math], simd complex math
using simd::real;
using simd::imag;
using simd::arg;
using simd::norm;
using simd::conj;
using simd::proj;
using simd::polar;


// See [simd.saturating.math], saturating math functions
using simd::saturating_add;
using simd::saturating_sub;
using simd::saturating_mul;
using simd::saturating_div;
using simd::saturating_cast;
```

§ 4.3. Add new section [**simd.saturating.math**]

Add the following section after [simd.complex.math].

🔗 `basic_vec` saturating math functions [simd.saturating.math]

```
template<vec-simd-type V> constexpr V saturating_add(const V& x, const V& y) noe
template<vec-simd-type V> constexpr V saturating_sub(const V& x, const V& y) noe
template<vec-simd-type V> constexpr V saturating_mul(const V& x, const V& y) noe
```

Constraints: The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Returns: A `basic_vec` object where the i^{th} element is initialized to the result of *`sat-func`*`(x[i], y[i])` for all i in the range `[0, V::size())`, where *`sat-func`* is the corresponding function from [numerics.sat.func].

```
template<vec-simd-type V> constexpr V saturating_div(const V& x, const V& y) noe
```

Constraints: The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Preconditions: For all i in the range of `[0..V::size())`, `y[i] == 0` is `false`.

Returns: A `basic_vec` where the i^{th} element is `saturating_div(x[i], y[i])` for all i in the range of `[0..V::size())`.

Remarks: If any `y[i] == 0`, the invocation is not a core constant expression.

```
template<class U, vec-simd-type V>
  constexpr rebind_t<U, V> saturating_cast(const V& v) noexcept;
```

Constraints: Both `U` and `typename V::value_type` are signed or unsigned integer types ([basic.fundamental]).

Returns: A `rebind_t<U, V>` where the i^{th} element is `saturating_cast<U>(v[i])` for all i in the range of `[0..V::size())`.

§ References

§ Informative References

[P0543R3]
Jens Maurer. *Saturation arithmetic*. 19 July 2023. URL: https://wg21.link/p0543r3